

计算机体系结构第二次实验报告

旁路前递

2312966 林晖鹏 计算机学院

一、实验要求

- 在第一次实验的五级流水线CPU的基础上，根据《CPU设计实战》这本书上第四章后面的内容，验证现有CPU的指令相关和流水线冲突问题，以添加旁路的方式和前递技术，让CPU规避这样的问题。

二、流水线冲突与阻塞分析

(1) 冲突阻塞概况

流水线冲突指指令在流水线各阶段（取指 IF、译码 ID、执行 EX、访存 MEM、写回 WB）流动时，因资源竞争或依赖关系导致后续指令无法按预期推进的现象，最终表现为“阻塞”，降低 CPU 吞吐量。

常见的三类冲突如下：

- 数据冒险**：后续指令需要使用前一条指令的运算结果，但前一条指令尚未完成结果写回，导致数据依赖无法满足。
- 控制冒险**：分支指令（如 BEQ）或跳转指令的目标地址需在执行阶段（EX）才能确定，而后续指令已提前进入取指阶段（IF），若分支跳转，已取指的指令需丢弃，产生“气泡”。
- 结构冒险**：多条指令在同一时钟周期竞争使用同一硬件资源（如寄存器堆、数据存储器），导致资源冲突。

(2) 避免冲突的设计

在实验一的源码中，已经设计了简单的延迟槽，以及一些阻塞来避免冲突。

1. 数据冒险的处理：

在 `decode.v` 中，通过 `rs_wait` 和 `rt_wait` 信号检测是否有数据冒险：

代码块

```

1  assign rs_wait = ~inst_no_rs & (rs!=5'd0)
2          & ( (rs==EXE_wdest) | (rs==MEM_wdest) | (rs==WB_wdest) );
3  assign rt_wait = ~inst_no_rt & (rt!=5'd0)
4          & ( (rt==EXE_wdest) | (rt==MEM_wdest) | (rt==WB_wdest) );

```

如果当前指令的源寄存器 `rs` 或 `rt` 依赖于 EXE、MEM 或 WB 阶段的结果，则会设置 `rs_wait` 或 `rt_wait`。在 `ID_over` 信号中，只有当 `rs_wait` 和 `rt_wait` 都为 0 时，ID 阶段才会完成：

代码块

```

1  assign ID_over = ID_valid & ~rs_wait & ~rt_wait & (~inst_jbr | IF_over);

```

2. 控制冒险的处理：

在 `pipeline_cpu.v` 中，通过 `cancel` 信号处理控制冒险：

代码块

```

1  wire cancel;    // 取消已经取到的指令，清空流水线

```

当发生跳转或异常时，`cancel` 信号会清空流水线，避免错误指令的执行。

`IF_allow_in` 信号控制 IF 阶段是否可以继续取指：

代码块

```

1  assign IF_allow_in = (IF_over & ID_allow_in) | cancel;

```

3. 流水线阻塞的处理：

在 `pipeline_cpu.v` 中，通过 `*_allow_in` 信号控制流水线的推进：

代码块

```

1  assign ID_allow_in = ~ID_valid | (ID_over & EXE_allow_in);
2  assign EXE_allow_in = ~EXE_valid | (EXE_over & MEM_allow_in);
3  assign MEM_allow_in = ~MEM_valid | (MEM_over & WB_allow_in);
4  assign WB_allow_in = ~WB_valid | WB_over;

```

只有当当前阶段完成（`*_over` 信号有效）且下一阶段允许接收指令时，流水线才会推进。

(3) 设计场景体现冲突

下面我们来设计一个场景来体现上面提到的冲突

代码块

```
1  # 初始: r1=5, r2=10, r3=0, r4=0, r5=0, r6=0, r7=0, r8=0, r9=0, r10=0, r11=0,
2  ADDU r3, r1, r2      # r3 = r1 + r2, r3=15, 后续指令依赖r3, 制造数据冒险
3  SUBU r4, r3, r1      # r4 = r3 - r1, 依赖上一条指令的r3
4  AND  r5, r4, r2      # r5 = r4 & r2, 依赖上一条指令的r4
5  LW   r6, 0(r5)       # r6 = MEM[r5], 依赖r5, 且为load-use冒险, 需暂停
6  ADDU r7, r6, r1      # r7 = r6 + r1, 紧跟load, 典型load-use阻塞
7  SW   r7, 4(r2)       # MEM[r2+4] = r7, 依赖r7, 继续冒险
8  BEQ  r7, r1, label   # 分支指令, 依赖r7, 制造控制冒险
9  ADDU r8, r1, r2      # 分支延迟槽, 正常执行
10 label:
11 ORI  r9, r0, 123     # 跳转目标, r9=123
12 # 乘法相关冒险
13 MULT r1, r2          # HI/LO = r1 * r2, 乘法多周期, 后续依赖HI/LO
14 MFLO r10             # r10 = LO, 紧跟MULT, 需等待乘法完成
15 MFHI r11             # r11 = HI, 紧跟MULT, 需等待乘法完成
```

下面这个是对应的COE文件（注释要删掉。）

代码块

```
1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  24010005, # addiu $1, $0, 5 // r1 = 5
4  2402000A, # addiu $2, $0, 10 // r2 = 10
5  24030000, # addiu $3, $0, 0 // r3 = 0
6  24040000, # addiu $4, $0, 0 // r4 = 0
7  24050000, # addiu $5, $0, 0 // r5 = 0
8  24060000, # addiu $6, $0, 0 // r6 = 0
9  24070000, # addiu $7, $0, 0 // r7 = 0
10 24080000, # addiu $8, $0, 0 // r8 = 0
11 24090000, # addiu $9, $0, 0 // r9 = 0
12 240A0000, # addiu $10, $0, 0 // r10 = 0
13 240B0000, # addiu $11, $0, 0 // r11 = 0
14 00221821, # addu $3, $1, $2 // r3 = r1 + r2
15 00612023, # subu $4, $3, $1 // r4 = r3 - r1
16 00822824, # and $5, $4, $2 // r5 = r4 & r2
17 8CA60000, # lw $6, 0($5) // r6 = MEM[r5]
18 00C13821, # addu $7, $6, $1 // r7 = r6 + r1
19 AC470004, # sw $7, 4($2) // MEM[$2+4] = r7
20 10E10001, # beq $7, $1, label // 分支冒险
21 00224021, # addu $8, $1, $2 // 分支延迟槽
22 3409007B, # ori $9, $0, 123 // 跳转目标
23 00220018, # mult $1, $2 // HI/LO = r1 * r2
```

```

24 000A502, # mflo $10 // r10 = L0
25 000B400; # mfhi $11 // r11 = HI

```

(4) 冲突阻塞分析

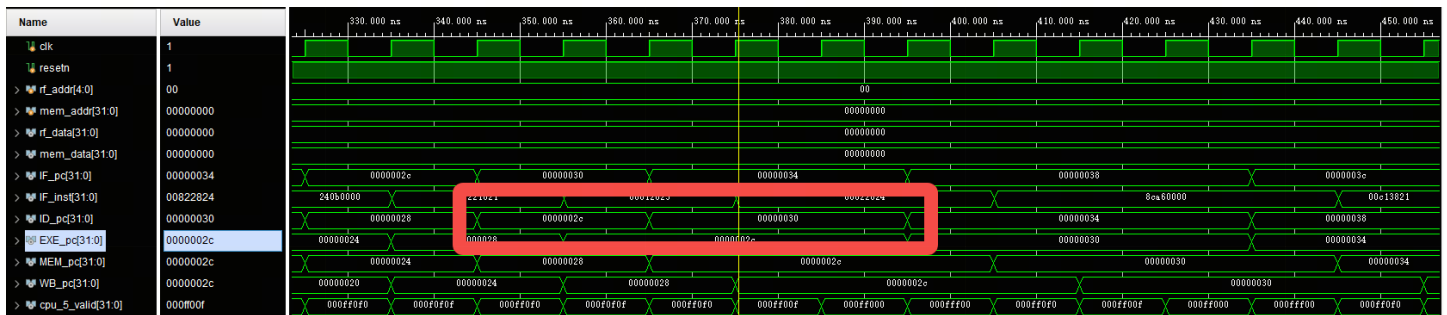
接下来我们分析一下我们上面这个场景中的冲突。

1. 数据冒险1-数据相关

pc为 0000002c 的指令对应 $r3 = r1 + r2$; pc为 00000030 的指令对应 $r4 = r3 + r1$ 。

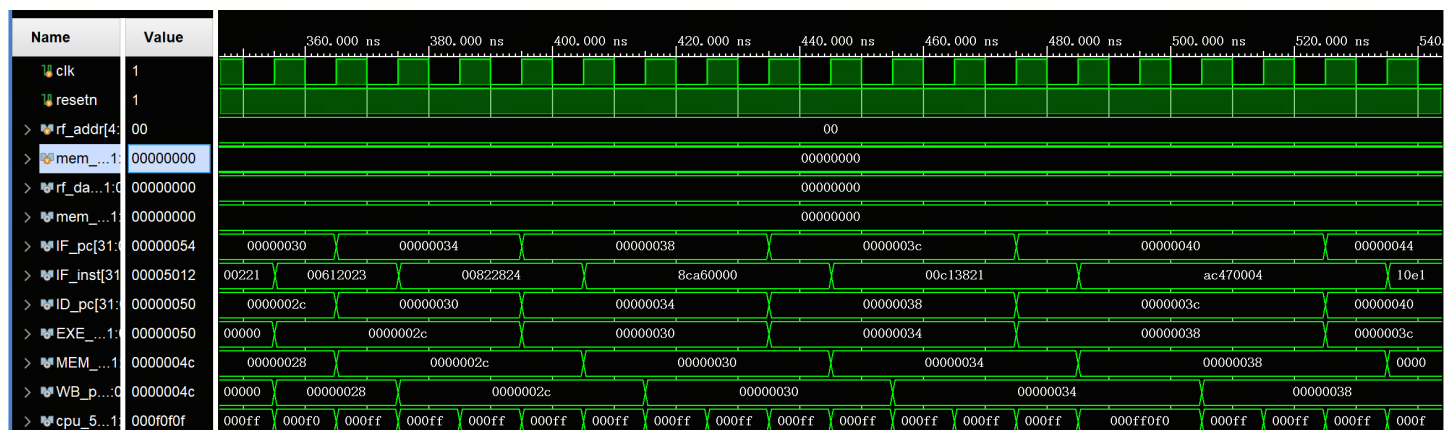
由于第二个指令需要用到寄存器r3的值，但是上一条指令还没写回r3，此时上文提到的wait信号就会阻塞第二条指令的译码阶段，直到r3被写回。

所以可以看到下面图中的第二条指令的译码阶段多阻塞了一个周期。



阻塞示意图

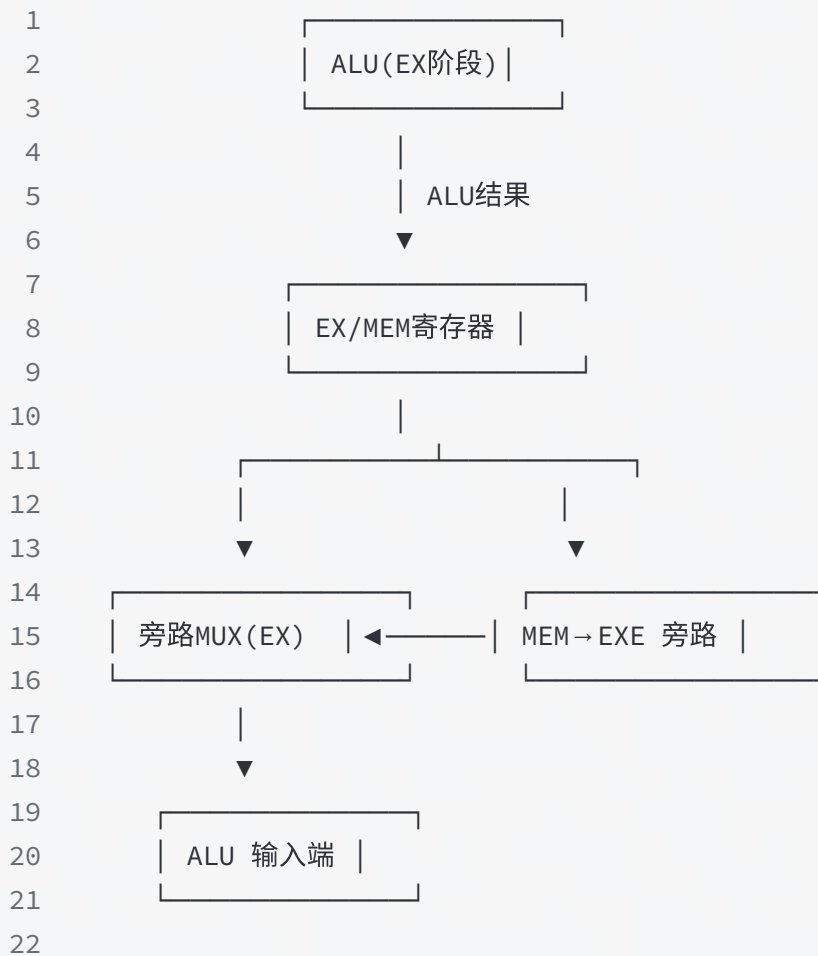
在上面场景的设计中，连续几条指令都连着出现了数据冒险，所以后面每个指令都被阻塞，占用周期的个数变多，导致流水线的效率变低。



三、旁路前递设计

(1) MEM、WB->EXE旁路

代码块



如果上一条指令的结果被下一条指令依赖，那么下一条指令需要等待直到上一条指令写回结果。

这里我们可以设计一个MEM->EXE\WB->EXE的旁路，让下一条指令直接进入EXE时拿到上一条指令EXE的结果，从WB或者MEM阶段传过来。

主要修改部分如下，**这里以MEM为例**：

- MEM、WB传回结果
 - 这里需要传回结果是否写能、结果的地址、结果的数据。用来和rs、rt对比，便于替换EXE中的操作数。
 - 然后就是对这些传回的结果进行赋值

代码块

```

1  //MEM
2  output          MEM_rf_wen_o,
3  output          [4:0]  MEM_wdest_o,
4  output          [31:0] MEM_forward_data_o,
5  ...
6  assign MEM_rf_wen_o      = rf_wen & MEM_valid & ~inst_store;
7  assign MEM_wdest_o      = rf_wdest;
8  assign MEM_forward_data_o = inst_load ? load_result : mem_result;

```

- pipeline传递结果

这里没有什么处理，只是把MEM、WB的结果简单传回去，不作赘述。

- EXE判断并处理操作数

- 首先是传入结果

代码块

```
1      input          MEM_rf_wen,
2      input      [4:0]  MEM_wdest,
3      input      [31:0] MEM_forward_data,
4      input          WB_rf_wen,
5      input      [4:0]  WB_wdest,
6      input      [31:0] WB_forward_data,
```

- 然后根据条件替换ID传入的原始操作数，这里值得注意的是，这里MEM的有限级是要比WB传回的结果的优先级是要高的，因为MEM的结果肯定是最终结果！

代码块

```
1  assign alu_operand1_actual = (MEM_rf_wen && (MEM_wdest == rs) && (MEM_wdest
    != 5'd0)) ? MEM_forward_data :
2                                     (WB_rf_wen && (WB_wdest == rs) && (WB_wdest !=
    5'd0)) ? WB_forward_data :
3                                     alu_operand1_raw;
4
5  assign alu_operand2_actual = (MEM_rf_wen && (MEM_wdest == rt) && (MEM_wdest
    != 5'd0)) ? MEM_forward_data :
6                                     (WB_rf_wen && (WB_wdest == rt) && (WB_wdest !=
    5'd0)) ? WB_forward_data :
7                                     alu_operand2_raw;
```

- decode译码阶段判断wait条件修改

- 此时部分指令就不需要再停留在ID阶段了，所以需要对 `rs_wait` 和 `rt_wait` 的定义进行修改。zh

代码块

```
1  assign rs_wait = ~inst_no_rs & (rs != 5'd0)
2                & ~(
3                (rs == MEM_wdest & MEM_wdest != 5'd0) // MEM阶段有rs的最新值
    (MEM→EXE旁路)
4                | (rs == WB_wdest & WB_wdest != 5'd0) // WB阶段有rs的最新值
    (WB→EXE旁路))
```

```

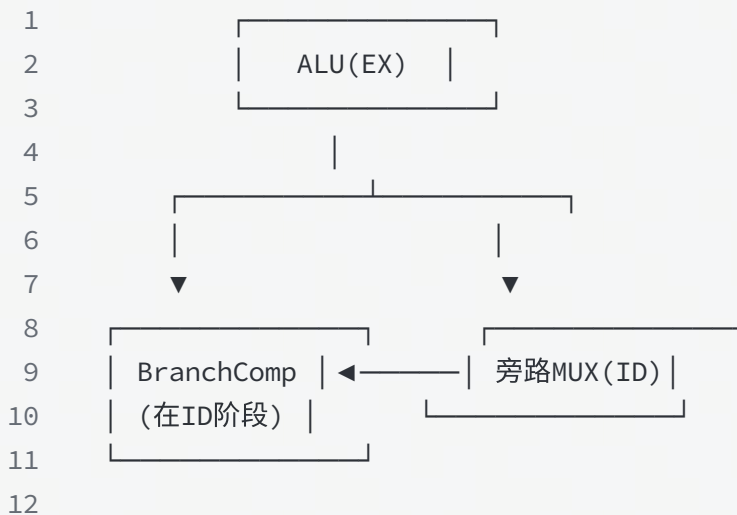
5         )
6         & (//其他原有条件, 这里不涉及修改
7         );
8
9     //rt同理

```

到这里，这两个旁路就添加完毕啦！

(2) EXE->ID旁路

代码块



如果第二条指令为跳转分支，由于跳转指令是在ID阶段进行计算的，如果依赖于上一条指令的结果，那么在ID部分将会被阻塞直到上一条指令的WB阶段结束。

这里我们设计一个EXE->ID的旁路，当EXE将结果计算完之后，传回ID阶段，让ID阶段提前计算出跳转的结果，这样就能防止阻塞到上一条指令到WB后才能计算跳转结果。

主要修改部分如下：

- EXE传出alu结果
 - 这里除了传出alu结果，有个坑就是如果这个指令是访存指令，那么ID要接受的数据需要等MEM结束之后才能拿到，所以这里需要做一个特殊判断，还需要传出此时EXE执行的指令是不是LOAD。

代码块

```

1   assign EXE_is_load = mem_control[3];
2
3   assign EXE_result = alu_result;

```

- pipeline传递结果，比较简单，这里不在赘述。
- ID阶段处理传入值
 - 这里传回的可能是rs也可能是rt，需要根据地址判断。
 - 然后更换跳转指令相关的赋值操作。

代码块

```
1  assign rs_forward_value = (rs == EXE_wdest && EXE_wdest != 5'd0) ?
   EXE_result : rs_value;
2  assign rt_forward_value = (rt == EXE_wdest && EXE_wdest != 5'd0) ?
   EXE_result : rt_value;
3
4  assign rs_equql_rt = (rs_forward_value == rt_forward_value);
5  assign rs_ez       = ~(|rs_forward_value);
6  assign rs_ltz      = rs_forward_value[31];
```

- ID阶段wait条件改变
 - 这里超级值得注意的是，如果EXE为乘法这种需要多个周期来完成的指令、或者LOAD这种需要在MEM阶段才能获得值的指令，这里仍然需要判断等待。

代码块

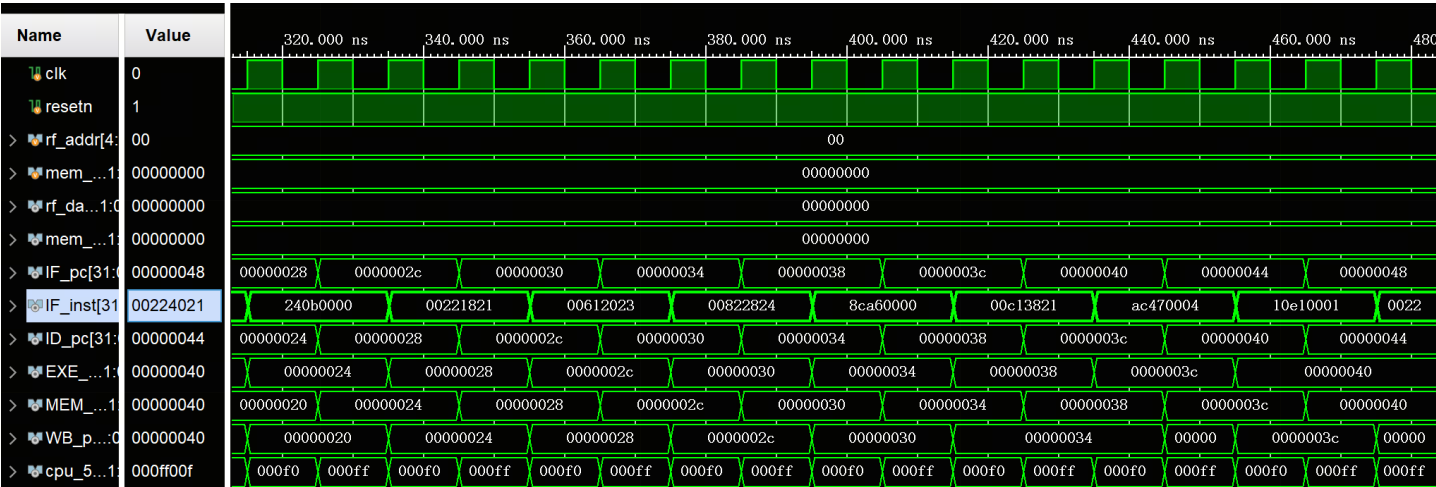
```
1  assign rs_wait = ~inst_no_rs & (rs != 5'd0)
2                & ~(
3                  (rs == MEM_wdest & MEM_wdest != 5'd0)
4                  | (rs == WB_wdest & WB_wdest != 5'd0)
5                  )
6                & (
7                  (rs == EXE_wdest & ~inst_jbr & EXE_wdest != 5'd0 &
   inst_MULT)
8                  | (rs == EXE_wdest & EXE_is_load)
9                  // | (is_exception & (EXE_valid | MEM_valid | WB_valid))
10                 );
```

到这里就大概改完这个旁路啦！

四、结果分析

1. MEM->EXE、WB->EXE旁路分析

在添加旁路之后，无论是前面算术指令造成的数据冲突还是后面的写回指令造成的数据依赖，冲突都已经消失，在我设计的场景中的前面连续的依赖指令都以两周期一个阶段的速度进行流水线，有效加快了流水线的效率！



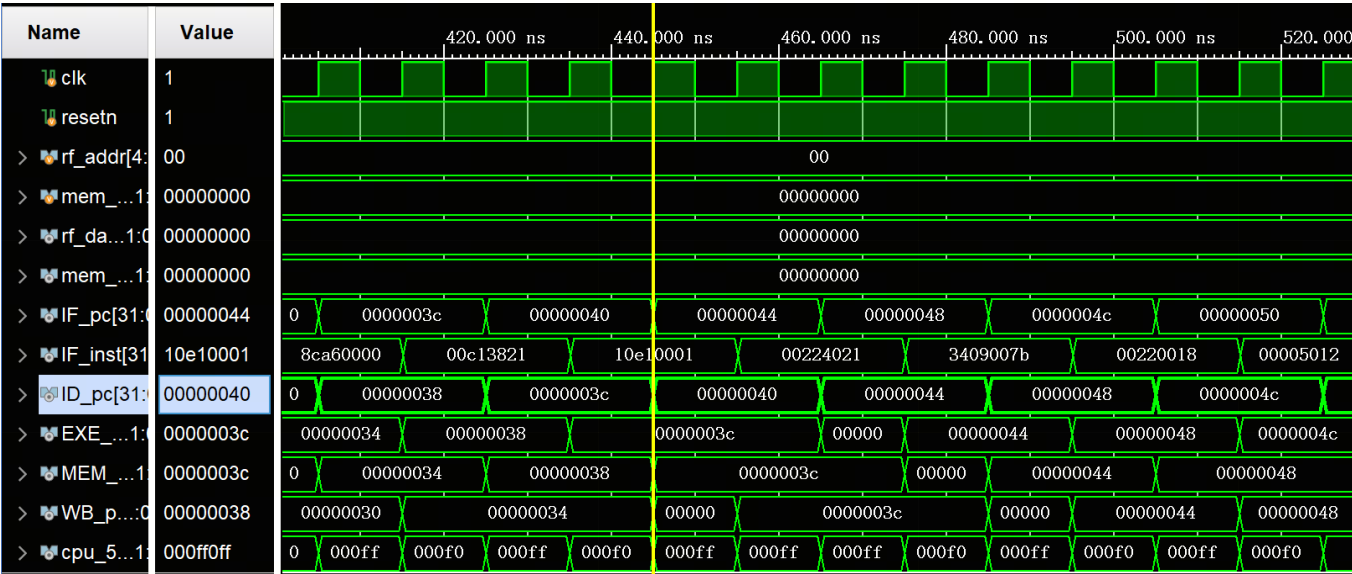
2. EXE->ID旁路分析

我们修改前面的场景，设置一个分支冒险出来：

代码块

```
1 00C13821, # addu $7, $6, $1 // r7 = r6 + r1
2 10E10001, # beq $7, $1, label // 分支冒险
```

在下图中pc=00000040时，指令为上面的跳转指令。可以关注到在修改完旁路之后，这里原本需要等到上一条指令到WB阶段才能结束ID阶段，但是并没有到WB，而是到MEM，也就是EXE结束后就进入下一个阶段了。



实验心得：

- 在完成旁路的时候总是对上次实验添加的新指令遗漏，在一些条件判断的时候缺少了考虑，比如中断指令和回复指令，导致做完旁路之后回来再想的时候感觉要改好多地方。无从下手。本质还是对上次实验的新指令不太熟悉。
 - 在实验过程中能想到一些指令无法周期内完成并前递，比如load指令，无法在EXE时获得结果；mult指令，无法一个阶段的时间就计算出结果，但是这里也是能考虑在减少一半时间的情况下前递已经计算完毕的低位部分的结果存入低位寄存器，然后给下一个指令赋值~！
-